

# Файл параметров RocksDB

[Прыжок на дно](#)Мерлин отредактировал эту страницу 1 сентября 2021 г. · [8 правок](#)

В RocksDB 4.3 мы добавили ряд функций, которые упрощают управление параметрами RocksDB.

1. Теперь каждая база данных RocksDB будет автоматически сохранять текущий набор параметров в файл при каждом успешном вызове функций `DB::Open()`, `SetOptions()` и `CreateColumnFamily()` / `DropColumnFamily()`.
2. [LoadLatestOptions\(\)](#) / [LoadOptionsFromFile\(\)](#): функция, которая создает объект параметров RocksDB из файла параметров.
3. [CheckOptionsCompatibility\(\)](#): функция, которая проверяет совместимость двух наборов параметров RocksDB.

Благодаря поддержке файлов параметров разработчикам больше не нужно поддерживать полный набор параметров ранее созданного экземпляра RocksDB. Кроме того, при необходимости изменения параметров функция `CheckOptionsCompatibility()` может дополнительно проверить, что полученный набор параметров позволяет успешно открыть ту же базу данных RocksDB без повреждения базовых данных.

## 🔗 Пример

Вот работающий пример, показывающий, как новые функции могут упростить управление параметрами RocksDB. Более полный пример можно найти в [examples/options\\_file\\_example.cc](https://github.com/facebook/rocksdb/blob/master/examples/options_file_example.cc).

Предположим, мы открываем базу данных RocksDB, создаем новое семейство столбцов прямо во время работы базы данных, а затем закрываем ее:

```
s = DB::Open(rocksdb_options, path_to_db, &db);
...
// Create column family, and rocksdb will persist the options.
```



```
ColumnFamilyHandle* cf;
s = db->CreateColumnFamily(ColumnFamilyOptions(), "new_cf", &cf);
...
// close DB
delete cf;
delete db;
```

Поскольку в RocksDB 4.3 и более поздних версиях каждый экземпляр RocksDB автоматически сохраняет свой последний набор параметров в файле параметров, мы можем использовать этот файл для создания параметров в следующий раз, когда захотим открыть базу данных. Это отличается от RocksDB 4.2 и более ранних версий, где для успешного открытия базы данных нужно было помнить все параметры для каждой группы столбцов. Теперь давайте посмотрим, как это работает.

Сначала мы вызываем `LoadLatestOptions()`, чтобы загрузить последний набор параметров, используемых целевой базой данных RocksDB:

```
ConfigOptions cfg_opts;
DBOptions loaded_db_opt;
std::vector<ColumnFamilyDescriptor> loaded_cf_descs;
LoadLatestOptions(cfg_opts, path_to_db, &loaded_db_opt, &loaded_cf_descs);
```



## 🔗 Unsupported Options

Since C++ does not have reflection, the following user-defined functions and pointer-typed options will only be initialized with default values. Detailed information can be found in `rocksdb/utilities/options_util.h`:

```
* env
* memtable_factory
* compaction_filter_factory
* prefix_extractor
* comparator
* merge_operator
* compaction_filter
```



For those un-supported user-defined functions, developers will need to specify them manually. In this example, we initialize Cache in `BlockBasedTableOptions` and `CompactionFilter`:

```
for (size_t i = 0; i < loaded_cf_descs.size(); ++i) {
    auto* loaded_bbt_opt = loaded_cf_descs[i].options.table_factory-
>GetOptions<BlockBasedTableOptions>());
```



```
loaded_bbt_opt->block_cache = cache;
}

loaded_cf_descs[0].options.compaction_filter = new MyCompactionFilter();
```

Now we perform sanity check to make sure the set of options is safe to open the target database:

```
Status s = CheckOptionsCompatibility(cfg_opts, kDBPath, db_options,
loaded_cf_descs);
```



If the return value indicates OK status, we can proceed and use the loaded set of options to open the target RocksDB database:

```
s = DB::Open(loaded_db_opt, kDBPath, loaded_cf_descs, &handles, &db);
```



## 🔗 Ignoring unknown options

In cases where an options file of a newer version is used with an older RocksDB version (say, when downgrading due to a bug), the older RocksDB version might not know about some newer options. `ignore_unknown_options` flag can be used to handle such cases. By setting the `ConfigOptions.ignore_unknown_options=true`, unknown options will be ignored. By default it is set to `false`.

## 🔗 RocksDB Options File Format

RocksDB options file is a text file that follows the [INI file format](#). Each RocksDB options file has one Version section, one DBOptions section, and one CFOptions and TableOptions section for each column family. Below is an example RocksDB options file. A complete example can be found in [examples/rocksdb\\_option\\_file\\_example.ini](#):

```
[Version]
rocksdb_version=4.3.0
options_file_version=1.1
[DBOptions]
stats_dump_period_sec=600
max_manifest_file_size=18446744073709551615
bytes_per_sync=8388608
delayed_write_rate=2097152
WAL_ttl_seconds=0
...
[CFOptions "default"]
```



```
    compaction_style=kCompactionStyleLevel
    compaction_filter=nullptr
    num_levels=6
    table_factory=BlockBasedTable
    comparator=leveldb.BytewiseComparator

compression_per_level=kNoCompression:kNoCompression:kNoCompression:kSnappyCompi
...
[TableOptions/BlockBasedTable "default"]
    format_version=2
    whole_key_filtering=true
    skip_table_builder_flush=false
    no_block_cache=false
    checksum=kCRC32c
    filter_policy=rocksdb.BuiltinBloomFilter
....
```

► **Pages** 167

## Contents

- [RocksDB Wiki](#)
- [Overview](#)
- [RocksDB FAQ](#)
- [Terminology](#)
- [Requirements](#)
- [Contributors' Guide](#)
- [Release Methodology](#)
- [RocksDB Users and Use Cases](#)
- [RocksDB Public Communication and Information Channels](#)
- [Basic Operations](#)
  - [Iterator](#)
  - [Prefix seek](#)
  - [SeekForPrev](#)
  - [Tailing Iterator](#)
  - [Compaction Filter](#)
  - [Multi Column Family Iterator](#)
  - [Read-Modify-Write \(Merge\) Operator](#)
  - [Column Families](#)
  - [Creating and Ingesting SST files](#)
  - [Single Delete](#)
  - [SST Partitioner](#)
  - [Low Priority Write](#)
  - [Time to Live \(TTL\) Support](#)
  - [Transactions](#)
  - [Snapshot](#)

- [DeleteRange](#)
- [Atomic flush](#)
- [Read-only and Secondary instances](#)
- [Approximate Size](#)
- [User-defined Timestamp](#)
- [Wide Columns](#)
- [BlobDB](#)
- [Online Verification](#)
- **Options**
  - [Setup Options and Basic Tuning](#)
  - [Option String and Option Map](#)
  - [RocksDB Options File](#)
- [MemTable](#)
- [Journal](#)
  - [Write Ahead Log \(WAL\)](#)
    - [Write Ahead Log File Format](#)
    - [WAL Recovery Modes](#)
    - [WAL Performance](#)
    - [WAL Compression](#)
  - [MANIFEST](#)
  - [Track WAL in MANIFEST](#)
- **Cache**
  - [Block Cache](#)
  - [SecondaryCache \(Experimental\)](#)
- [Write Buffer Manager](#)
- [Compaction](#)
  - [Leveled Compaction](#)
  - [Universal compaction style](#)
  - [FIFO compaction style](#)
  - [Manual Compaction](#)
  - [Subcompaction](#)
  - [Choose Level Compaction Files](#)
  - [Managing Disk Space Utilization](#)
  - [Trivial Move Compaction](#)
  - [Remote Compaction](#)
- **SST File Formats**
  - [Block-based Table Format](#)
  - [PlainTable Format](#)
  - [CuckooTable Format](#)
  - [External Table](#)
  - [Index Block Format](#)
  - [Bloom Filter](#)
  - [Data Block Hash Index](#)
- [IO](#)
  - [Rate Limiter](#)
  - [SST File Manager](#)
  - [Direct I/O](#)
- [Compression](#)
  - [Dictionary Compression](#)
- [Full File Checksum and Checksum Handoff](#)
- [Background Error Handling](#)

- [Huge Page TLB Support](#)
- [Tiered Storage \(Experimental\)](#)
- **Logging and Monitoring**
  - [Logger](#)
  - [Statistics](#)
  - [Compaction Stats and DB Status](#)
  - [Perf Context and IO Stats Context](#)
  - [EventListener](#)
- [Known Issues](#)
- [Troubleshooting Guide](#)
- [Tests](#)
  - [Stress Test](#)
  - [Fuzzing](#)
  - [Benchmarking](#)
- **Tools / Utilities**
  - [Administration and Data Access Tool](#)
  - [How to Backup RocksDB?](#)
  - [Replication Helpers](#)
  - [Checkpoints](#)
  - [How to persist in-memory RocksDB database](#)
  - [Third-party language bindings](#)
  - [RocksDB Trace, Replay, Analyzer, and Workload Generation](#)
  - [Block cache analysis and simulation tools](#)
  - [IO Tracer and Parser](#)
- **Implementation Details**
  - [Delete Stale Files](#)
  - [Partitioned Index/Filters](#)
  - [WritePrepared-Transactions](#)
  - [WriteUnprepared-Transactions](#)
  - [How we keep track of live SST files](#)
  - [How we index SST](#)
  - [Merge Operator Implementation](#)
  - [RocksDB Repairer](#)
  - [Write Batch With Index](#)
  - [Two Phase Commit](#)
  - [Iterator's Implementation](#)
  - [Simulation Cache](#)
  - [\[To Be Deprecated\] Persistent Read Cache](#)
  - [DeleteRange Implementation](#)
  - [unordered\\_write](#)
- **Extending RocksDB**
  - [RocksDB Configurable Objects](#)
  - [The Customizable Class](#)
  - [Object Registry](#)
- **RocksJava**
  - [RocksJava Basics](#)
  - [Logging in RocksJava](#)
  - [JNI Debugging](#)
  - [RocksJava API TODO](#)
  - [RocksJava Performance on Flash Storage](#)

- [Tuning RocksDB from Java](#)
- **Performance**
  - [Performance Benchmarks](#)
  - [In Memory Workload Performance](#)
  - [Read-Modify-Write \(Merge\) Performance](#)
  - [Delete A Range Of Keys](#)
  - [Write Stalls](#)
  - [Pipelined Write](#)
  - [MultiGet Performance](#)
  - [Tuning Guide](#)
  - [Memory usage in RocksDB](#)
  - [Speed-Up DB Open](#)
  - [Implement Queue Service Using RocksDB](#)
  - [Asynchronous IO](#)
  - [Off-peak in RocksDB](#)
- [Projects Being Developed](#)
- **Misc**
  - [Building on Windows](#)
  - [Developing with an IDE](#)
  - [Open Projects](#)
  - [Talks](#)
  - [Publication](#)
  - [Features Not in LevelDB](#)
  - [How to ask a performance-related question?](#)
  - [Articles about Rocks](#)

## Clone this wiki locally

`https://github.com/facebook/rocksdb.wiki.git`

